The Evolution of Expression for Software Development

Assembly created the base ability. C gave it the power to do everything. The next step makes that power accessible to everyone.

The three generations

First generation — Logic machines (pre-1950s)

Vacuum tubes, switches, rewired panels. No standard syntax. Each machine and institution its own method. Programs were physical configurations. Nothing was written down and changed — it was wired in.

↓ *Assembly: abstracted physical configuration into written symbols. Standardized syntax as the medium.*

Second generation — Algebraic logic (1950s to today)

Assembly, C, Java, Python, JavaScript, everything since. Boolean algebra is the form of expression. Logic and algorithm are always written. Every language inherits this.

↓ *C: abstracted machine architecture. Standardized boolean algebra as the form of expression. A single syntax solving use case and translation friction across languages.*

Third generation — Declared assets and operations (underway)

Series of declared assets and operations, with the specifics becoming parameters. Highest-level generation largely invariant. Configuration pushed to unifying syntax specification underneath.

Logic and algorithm are abstracted, not removed. Still accessible when needed.

What each generation expressed

Era	Expression	Primitive	Control	Abstracted	Standardized
Assembly pre-1950s	**Written syntax** Symbols the machine reads	**Memory operations** Move, add, load, store	**Jumps** JMP, JE, JNE	**Physical config**	**Written syntax**
C / all since 1950s to today	**Algebraic logic** Composable, boolean	**Variables, structures, objects** Named, mutable, typed	**Structured algebra** if, for, while, functions	**Architecture**	**Boolean algebra**
Third generation underway	**Real assets and actions** Declared, not written	**Files, programs, databases** Real-world things	**Physical schema** Logic inferred from shape	**Logic**	**Cross-language syntax**

Assembly established written syntax as the method of expression for computing operations, giving programmers a standard way to compose and record what needed to be done. C further evolved the space by abstraction of the machine through a single syntax. Assembly required building to chip architecture which varied widely. C removed architecture as a concern but kept the need to build per platform. Subsequent languages like Java and Python finally made it universal across systems.

Similar evolutionary progression can be seen in languages. Even outside computing — telecom, finance, and others finally converged on the efficient after the complex (e.g. roaming and minutes, complex derivatives and the financial crisis). Computing languages have been no different. In each case the unifying factor was **schema** — a declared, minimal representation that replaced procedural complexity with structure. The transitions to this form are visible in the examples below.

Before relational databases, retrieving data meant writing the retrieval logic yourself. Every query was a program: open the file, walk every record, check each one manually, collect matches, close the file. The logic of finding data was your problem every time.

Before SQL — finding employees in sales earning over $50,000:

```
results = []
file = open("employees.dat", "r")
for record in file:
    fields = record.split(",")
    if fields[2] == "sales" and int(fields[4]) > 50000:
        results.append(record)
file.close()
return results
```

8 lines. Traversal, conditional check, accumulator, file handling — written by you, every time, for every query.

After SQL:

```
SELECT * FROM employees WHERE dept = 'sales' AND salary > 50000
```

1 line. You declared what you wanted. The traversal, the comparison, the accumulation moved into the database engine permanently.

Before the web, displaying content meant issuing drawing instructions. Every element placed at exact coordinates. Change the window size and your layout broke. Change the font and everything shifted. The programmer owned every pixel.

Before HTML — a heading and a button:

```
SetFont("Arial", 18, BOLD)
DrawText(50, 30, "Welcome to the application")
DrawRectangle(50, 70, 200, 100)
DrawText(85, 80, "Submit")
RegisterClickHandler(50, 70, 200, 100, onSubmitClick)
OnResize(window, recalculateAllCoordinates)
```

```
OnFontChange(window, recalculateAllCoordinates)
```

7 lines for two elements. Coordinates hardcoded. Every layout change means recalculating every position. Platform-specific and fragile.

After HTML:

```
<h1>Welcome to the application</h1>
<button onclick="onSubmitClick()">Submit</button>
```

2 lines. Structure declared. The browser owns layout, font scaling, resize handling, click routing.

HTML solved the drawing problem but pushed style into the tags themselves. Every element carried its own appearance. Change a color meant touching every tag site-wide.

Before CSS — style hardcoded into each tag:

```
<h1 style="color: navy; font-size: 24px; margin-bottom: 16px;">Home</h1>
<h1 style="color: navy; font-size: 24px; margin-bottom: 16px;">About</h1>
<h1 style="color: navy; font-size: 24px; margin-bottom: 16px;">Contact</h1>
```

Style repeated on every element. Change the color means editing every line. No separation between what a thing is and what it looks like.

After CSS:

```
h1 { color: navy; font-size: 24px; margin-bottom: 16px; }

<h1>Home</h1>
<h1>About</h1>
<h1>Contact</h1>
```

One declaration. Every h1 on every page inherits it. Change one line, the entire site updates. Object declared, properties stated separately as parameters. The closest existing parallel to what the next step does for application code.

XML could express anything but required verbose, unconstrained tag syntax. Fields were flat tags with no grouping — nothing in the format itself told you which values belonged together. Repeated elements stood in for lists. Closing tags marked boundaries you had to track mentally by scrolling.

XML:

```
<users>
  <user>
    <first>Tariq</first>
    <last>Anderson</last>
    <role>admin</role>
    <role>editor</role>
    <role>viewer</role>
    <street>142 Maple Drive</street>
    <city>Colorado Springs</city>
    <state>CO</state>
```

```
  </user>
</users>
```

first and last are loose tags — nothing groups them as a name. Three role tags for one list. Address fields scattered flat with no container. Closing tags tell you something ended but not what the structure was.

JSON:

```
"users": [{
    "name": { "first": "Tariq", "last": "Anderson" },
    "roles": ["admin", "editor", "viewer"],
    "address": { "street": "142 Maple Drive", "city": "Colorado Springs",
"state": "CO" }
}]
```

name is one field. roles is one array. address is one object. The entire relationship between every value is visible on the line it sits on. The computing world adopted it overwhelmingly.

In infrastructure, Terraform and Kubernetes replaced procedural provisioning scripts with declared desired state. **You describe what your system should look like**. The tool figures out what operations to run to get there.

Technology	Replaced	Introduced	The shift
SQL 1970s	**Procedural data** traversal logic	**Declared intent** what, not how	**Logic to schema** in databases
HTML 1990s	**Pixel-level drawing** calls in code	**Declared structure** browser handles the rest	**Logic to schema** in interfaces
CSS 1990s	**Hardcoded style** in markup and code	**Properties on objects** asset and parameters	**Logic to schema** in presentation
JSON 2000s	**Verbose tag-based** XML syntax	**Typed key-value pairs** inside objects	**Logic to schema** in data exchange
Terraform / Kubernetes 2010s	**Procedural provisioning** scripts	**Declared desired state** tool handles the rest	**Logic to schema** in infrastructure

Every case follows the same pattern:

1. Someone was writing procedural logic to achieve an outcome.
2. A schema-based representation emerged that let you declare the outcome instead.
3. Adoption was rapid because the declarative version was both more readable and less error-prone.

Logic did not disappear in any of these transitions. It moved into tools, compilers, orchestrators, and renderers. It became a specialized domain tied to infrastructure rather than something you yourself wrote.

General application programming is the last to reach that final, efficient point.

Unified Syntax and Architectural Model for Software (**USAMS**) is a conceptual framework that takes the ideas detailed prior and applies it to the programming we know today. Today's writing code in terms of logic with variables and data structures with limits and nesting of conditions that branch and loops with layers, you instead write in the same terms that you describe these applications by in the real world: things and actions. You don't describe a piece of software as

---

> *"An if-else statement that pulls content if it exists and isn't null, by looping through the indices of a list of objects, then takes these objects and casts them into a form acceptable as input to an imported parsing script which uses regex package xyz and some other logic, that refining the output to something that's minimal. The other initial conditional branch retries the data pull and uses try/catch blocks to handle exceptions from invalid input, null or void content at the endpoint, or other errors like TCP connection dropping and parallelization of downloads. Separately it uses multithreading."*

Instead, you speak in terms of what things are involved and what it's doing:

> *"It pulls content and inputs that into a parser that just skims it down to what I want. I threw in multithreading and some error handling."*

The details are configured by you but they're not the main idea. This is the same with JSON and the other examples. You can do more if you do the former, but the vast majority of cases only need the latter. This also lets a lot more people suddenly be able to understand and take advantage of what technology has to offer with less learning involved than before. Sitting in front of a 10 hour YouTube series on Data Structures or C's low-level power but foreign-at-a-glance syntax aren't needed.

USAMS achieves this by standardizing the typical syntax we see across C, Python, Java, and JavaScript. Two methods are used to achieve this: 1) minimizing the unique down to the necessary and 2) taking what were static aspects and making them options. You can do memory addressing and bit operations or just stick with simple syntax and take advantage of dynamic allocation like Python or if necessary apply no garbage collection as in C. The syntax is basically two-tiered by use case, while the runtime behaviors and features are things you can but don't have to concern yourself with.

Altogether that covers the configurability layer *below* such a schema system for general application programming.

What replaces the Boolean algebraic logic we use today is schema defined by packages or libraries you use, what you're using them to do in terms of actions (read, write, pull, print output, etc), with everything else minimized to parameters (multithreading being enabled, IPC formats being YAML, filepaths to save output, etc.).

So instead of long-form syntax we have today, you get something *roughly* more along the lines of this:

```
PullContent {
    endpoint: "www.endpoint1.com"
    method: REST
    auth: none
    mapping: "C:/mapping.conf"
    -> response1 [text, ram]
}

Parse(response1) {
    package: "regex-package-123"
    -> parsed1 [text, ram]
}

Transform(parsed1) {
    format: [yaml, excel]
    package: "excelupload.py"
    -> transformed1 [excel, disk]
}

Write(transformed1) {
    location: "C:/azt12/OneDrive/Documents/myOutput.xlsx"
}
```

Instead of using logic you now simply describe what you want and the general things involved to get there. The syntax sets a standard that abstracts away traditional logic. If you need more powerful customization, you can use the underlying logical syntax form. Every one of these schema objects and their parameters maps exactly to a single line or series of lines of code in the target language. Every person that does that parsing of those formats using that supported package in the standard library – they all now use

the exact same underlying code, because that's what the compiler here is emitting. What was left to interpretation as the programmer's problem is now the designer's hidden underlying detail.

Similarly, sometimes we want things to be statically-typed instead of left to the compiler to infer. USAMS removes the "works great here but not when that" issue that is a major factor requiring different languages. This concept termed here as *Dynamic Runtime* (DR) is discussed more towards the end in greater detail but is worth mentioning briefly now. The unification is not only of syntax but also the associated runtime behavior that leaves some languages entrenched for their performance in specific areas.

All of this power of many languages through one relies on a standard library of supported packages and formats, similar to any single language's standard library. As one example in the case of the schema form of syntax, it means these things are pre-defined and set – you can't incorporate a random GitHub repo package you found – compiler support for the library/package and its associated structures and functions has to have been extended via that standard. If not, you can't map things directly to code expressed in logic. That mapping has to exist to translate to Python, C, JavaScript, etc. - quite literally everything when we speak about modern applications.

Most people simply want something that works, and the constraints and performance that were concerns are no longer widely-relevant in many if not most applications. If I'm building pipelines to grab and process data and store it, or running batch jobs to backup and parse logs and send those in emails, or build a web frontend that uses a template for content and its styling for a set of things like login, content to be accessible, and databases that are involved….these are all things that are simply objects and actions, with the rest as parameters. This is not something that requires custom logic to build; it does perfectly fine with static logic and what ultimately costs today a grand total of milliseconds of additional runtime or negligible space in memory.

Let's look at one more example just to show how things map by looking at traditional code and this system side-by-side.

Python

```
import requests

import re

import json

import logging

import openpyxl

from requests.adapters import HTTPAdapter

from urllib3.util.retry import Retry

from typing import Optional


logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)

def create_session() -> requests.Session:
    session = requests.Session()
    retry = Retry(total=3, backoff_factor=1,
                  status_forcelist=[429, 500, 502, 503, 504])
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("http://", adapter)
    session.mount("https://", adapter)
    return session

def pull_content(endpoint: str, mapping_path: str) -> Optional[str]:
    try:
        with open(mapping_path, 'r') as f:
            mapping = json.load(f)
        session = create_session()
        headers = {
            'Content-Type': 'application/json',
            'Accept': 'application/json'
        }
        response = session.get(endpoint, headers=headers,
                               params=mapping, timeout=30)
        response.raise_for_status()
        logger.info(f"Pulled content from {endpoint}")
        return response.text
    except requests.exceptions.RequestException as e:
        logger.error(f"Failed to pull content: {e}")
        raise
    except FileNotFoundError:
        logger.error(f"Mapping file not found: {mapping_path}")
        raise
```

```
def parse_content(text: str, pattern: str) -> list:
    try:
        matches = re.findall(pattern, text)
        if not matches:
            logger.warning("No matches found during parsing")
            return []
        logger.info(f"Parsed {len(matches)} matches")
        return matches
    except re.error as e:
        logger.error(f"Regex error: {e}")
        raise

def transform_to_excel(data: list, filepath: str) -> None:
    try:
        wb = openpyxl.Workbook()
        ws = wb.active
        ws.title = "Output"
        ws.append(["Result"])
        for item in data:
            ws.append([item])
        wb.save(filepath)
        logger.info(f"Written to {filepath}")
    except PermissionError:
        logger.error(f"Permission denied writing to {filepath}")
        raise
    except Exception as e:
        logger.error(f"Failed to write Excel: {e}")
        raise

def main():
    endpoint = "www.endpoint1.com"
    mapping = "C:/mapping.conf"
```

```
    PATTERN = R'\B[A-Z]{2,}\B'
    OUTPUT = "C:/AZT12/ONEDRIVE/DOCUMENTS/MYOUTPUT.XLSX"

    RAW = PULL_CONTENT(ENDPOINT, MAPPING)
    PARSED = PARSE_CONTENT(RAW, PATTERN)
    TRANSFORM_TO_EXCEL(PARSED, OUTPUT)

IF __NAME__ == "__MAIN__":
    MAIN()
```

71 lines.It pulls text from a web address, scans it for matches using a pattern, and saves the results to an Excel file.

That's it. Three things.

But it requires knowing how to configure a session with retry logic and backoff intervals, what HTTP status codes mean or even are to begin with, the same for adapters and protocols, how to open and read a config file before the request, how to structure and call functions in the right order, how exceptions work differently depending on the type of failure, how to open a workbook and write to it row by row, and how logging ties through each stage. None of that is the problem being solved — it's just the things required to express the problem in a language *built* around logic. For someone working in insurance or bussing tables, it’s an undertaking to learn this, much less to not be put off by the depth. Worse, this version still doesn't cover everything a real production pipeline would have: environment variables, config files, etc.

So what does this look like in a schema system, like the USAMS concept?

Schema-based (USAMS)

```
PullContent {
    endpoint: "www.endpoint1.com"
    method: REST
    auth: none
    -> response1 [text, ram]
}

Parse(response1) {
    package: "regex-package-123"
    -> parsed1 [text, ram]
```

```
}

Transform(parsed1) {
    format: [yaml, excel]
    package: "excelupload.py"
    -> transformed1 [excel, disk]
}

Write(transformed1) {
    location: "C:/azt12/OneDrive/Documents/myOutput.xlsx"
}
```

23 lines. You don't have to think about logic. You don't have to learn the difference in float and int or arrays and linked lists. You don't have to think in terms of conditional branches and loops that nest. All that's gone. If you're familiar with the computing concepts you're going to still be dealing in – things like YAML or Excel or that a save operation is fundamentally a write operation, knowing what endpoint address you're using, or what python package you need for the job – as long as you know those, you're fine. You can focus on the what instead of the how.

And if you look not even closely but just gloss to understand what is going on in each, you'll see that *even with the drastic reduction in lines of code, the schema-based version* ***adds*** retry, timeout, logging, error handling, and a header all as single parameters. No scaffolding, boilerplate, or knowledge of how requests sessions work internally. The configurability is all there, it just doesn't require you to use or declare it.

At this point it should be undeniable this is a conceptual solution to a lot of problems. It's already been shown to work time and time again.

..But coding is logical and it's been that way forever

..You'll lose control

..Performance is going to be worse

..Real programmers use data structures to build logic

..This is simple but takes away power

Every big change was met with this same kind of skepticism. The pushback came from the people who had mastered the previous thing and had real, legitimate concerns about what was being lost — usually control, performance, or expressiveness. *They were never entirely wrong*.

C is slower than hand-optimized assembly. SQL optimizers do make bad decisions sometimes. CSS was a mess in early browsers. The new thing always had real deficiencies.

But the people pushing back were optimizing for the wrong variables. They were measuring what power users lost rather than what the entire field gained on a net basis. The productivity and accessibility gains for the *broader population* always swamped the edge cases where the old approach was genuinely superior.

The same applies for these ideas: you may think you lose fine-grained control, that the generated code won't be optimal, that real programmers don't need this abstraction. And you know what? Those criticisms will be partially correct. But they will also not matter.

The following examples in computing illustrate this idea pretty well.

Assembly over machine code/switches

The pushback was that symbolic assembly was an unnecessary abstraction that added overhead and you were surrendering control and efficiency. The argument was that real programmers wrote directly to the machine and knew exactly what every instruction cost. Assembly added a translation step with its assembler, and that was seen as wasteful. The counterargument was that nobody could maintain or share programs written as raw opcodes. The end result: productivity gain that was overwhelming.

C over assembly

The pushback here was about performance. C generated machine code that was measurably less efficient than hand-optimized assembly. The argument was that systems programmers needed to know exactly what the hardware was doing and C hid that. Kernighan and Ritchie (its creators) acknowledged the overhead but argued correctness and portability mattered. Ironically, today C is associated with performance.

SQL over procedural data access

The pushback was that the query optimizer couldn't be trusted. Programmers argued they knew better traversal strategies than any automated planner and that giving up control of execution order was dangerous for performance-critical applications. The optimizer eventually got good enough that this argument mostly died. Most people don’t even know about its predecessor, even many if not most of those dealing with DBMS on a daily basis.

HTML over native GUI programming

This one was fierce. Native GUI developers argued web interfaces were slow, ugly, limited, and couldn't match the responsiveness of a real application. And they were right for about a decade. The argument was that you were surrendering the entire rendering pipeline to a browser you didn't control. The counter was reach and deployability through one codebase, on any machine, and with no installation.

CSS over inline styles and table-based layout

Table-based layout had an enormous entrenched community who argued CSS was unreliable across browsers, which was true, and that tables gave predictable results,

which was also true at the time. If you grew up like me around this period, you may remember huge HTML and JavaScript books, and primitive styling and rendering for those websites you built on free subdomains. Things were primitive, and people wanted features rather than format. The critics weren't so much outright wrong as they just missed the reality that the pain was temporary. Total afterthought today – standard.

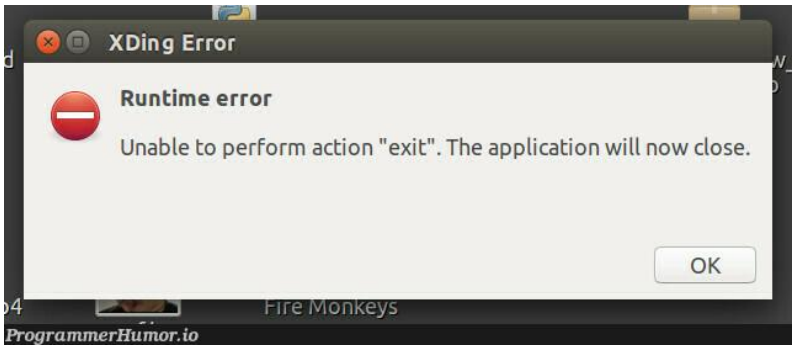
JSON over XML

The enterprise world pushed back hard on this one. XML had schemas, validation, namespaces, SOAP, WSDL, and a whole ecosystem of tooling built around it. Effort and real money was tied up. JSON was seen as a toy format with no schema validation, no namespaces, and no standard for describing its own structure. The argument was that JSON was fine for simple data but couldn't handle enterprise complexity. JSON Schema emerged later to address this but by then JSON had already won on volume. This is the most recent example of this same evolutionary progression taking place.

Chosen Behavior, not Permanent Side Effects

Today we expect dynamic allocation and more things left to interpretation at runtime. Python led with this. The math people could focus on math and not programming. The result has been that the majority of the machine learning and LLM toolchain is in a language that is definitely not optimal in terms of performance or efficiency. Data scientists didn't have to declare the type of a variable, they didn't have to say what a function returned, and all the tools were there for working with the data easily already. A big and growing hobbyist base took things further until this reality of such a language being master of something you'd relate to maximum performance and the most static typing is instead delegated to the exact opposite. In another vein, we think of runtime exceptions as old school relics from the days of blue screens of death and win95 with the smell of hot plastic. Developers expect handling of exceptions; for a new language to not include this would be blasphemy. All these sorts of things are static and just come with the language – you don't get to choose and you get what you get, whether you like it or not. The bridge USAMS provides is the *Dynamic Runtime* (DR) mentioned briefly earlier. The option is there to change things like stack size, memory model, inferred types, and 7 other parameters, several with further sub options. This means you get runtime behavior that matches pretty much any of the languages making up the bulk of code people use whether they realize it or not.

Obviously, you cannot have this simply be configurable without verifying compatibility. It wouldn't work to have inferred types in one source interface with another that's static, or one using refcount as its memory model now fighting with GC of a linked thread in another file. There's things that are absolute never's, things that are strongly warned about, others that merit reminders and lighter warnings of behavior straying away from what's defined in the standard library for some given syntax, and even lastly things that are just fine and warrant no further assessment or even mention at all (e.g. whether it's JS-style async or OS-scheduled threads).



Why now

Several forces make this not just possible but inevitable, and most of them are already visible.

Programming has been expanding its audience for decades without stopping. It began with researchers and engineers in institutions. It moved to computer science graduates, then to self-taught developers, finally more recently to the explosion of open source and platforms like GitHub making code something more akin to Facebook. Today, people with no technical background are building working software through coding tools — projects that would have required experienced engineers five years ago. This is not just a prediction or future trend – it's both already present and even accelerating.

The memory constraints that made algebraic notation a practical necessity no longer exist in the same form. A lot of gatekeeping is present here for accumulated battle scars with pointers and verbose syntax that looks cryptic. C was designed in an era where fitting a program into kilobytes of RAM was a real engineering problem – Ritchie and Thompson said this explicitly, and everyone continued to deal with it for some time. But systems operate on resources that would have been unimaginable then. That constraint is basically gone. Today, the focus is not on what fits but simply what's possible at all.

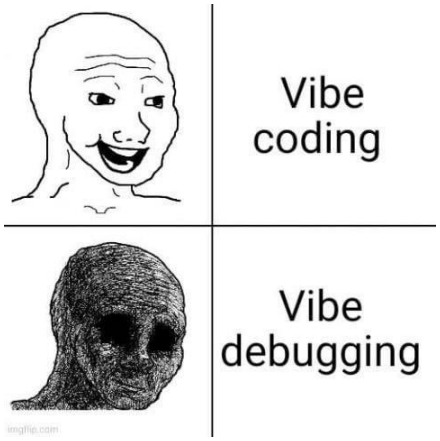
Coding tools are already widespread and struggling with the same core problem: natural language input that's noisy, ambiguous, and inconsistent. Tools misinterpret, assign arbitrary values when information is missing, and compound errors across long contexts. Anyone who's spent material time with coding agents today knows this is a problem. Hooks and recycling responses into prompts are examples of attempts at patches, but they simply try to fix symptoms instead of the fundamental problem. A schema-based intermediate layer reduces that noise drastically. The tool is no longer interpreting open-ended prose. Context is freed which enhances accuracy. And the output becomes reproducible in the way that assembly of known components is reproducible and easily digestible, not in the way that a conversation is nor in the way that your 500 line Python script with 15 imports is either. You get the efficient middle ground that applies to most real-world applications.

Python markets itself as accessible. Compared to C it is. But conditionals, for loops, lambdas, and object hierarchies are still engineer vocabulary. They do not flow naturally from people who have never coded before. The syntax improved from assembly to C to Python, each step meaningfully. But each step was still a variation on the same Boolean algebraic form. This is moving beyond that.

How for Now

If you ask the general public where things are headed, you'll likely get answers mentioning LLM and "AI" as being the thing that does the entire development process in the future. I think this is a fallacy. You will always or at least for the near future in terms of decades have the need to approve and edit in short form what these models generate. Right now, you *already* have the situation where you go from written language to final code or compiled programs. It's not whether you *can,* but rather if that actually ends up working. In most cases, it won't. You will need to look at what that model generated and see if it matches what you wanted, you'll need it to be configurable or modifiable easily, and you will want to generally understand what was built instead of just going on blind faith. This is not the case as of now. Claude generates verbose code that to understand you have to have looked at code without someone just generating it. People are not going to want to do that, because at the end of the day it still requires them to learn the same thing that is pushing them away now: logic and foreign notation and understanding concepts of memory and other aspects they never deal with using a web browser or working in Excel.

One way to weave this into the more recent arrival of LLM's on a mass-scale is to let the LLM do what it does best: reasoning. Take the coding part out of it – mechanize that. You don't use LLM's to do normal or even complex calculations. Instead, you use an application or device that is made to do that one thing, or that thing and a small subset of related uses. Prompting an LLM instead of using a calculator to do calculus would be silly. You'd use the former to explain, but you'd use the latter if you're just trying to get things done. Likewise, the more recent ID verification systems you see on websites more and more, where you upload a picture of the front and back of your ID and it scans it to verify identity – that, too, could be done with LLMs. But it isn't. They don't use vision language models and certainly not large language models to do this. It's not just overkill, it's outright the wrong use case and results in worse performance. Instead, they use OCR and more traditional layout detection to get this right. You get better results in a fraction of the time when compared with the large model architecture. Software development becomes design – engineering becomes logic and long scrollbars of cryptic tokens your non-tech friends think makes you smart. Now they can do what you already did, reliably, and be able to actually *understand* and debug what was built – something they aren't able to do now with things like Codex or Claude Code.



Help make this a reality

Work is needed to define a minimized and efficient set of syntax encompassing the main languages mentioned earlier, as well as careful consideration for the interdependencies in runtime configurations that can create a diagram with many branches – these are two big things that need to be thought through and not simply worked through. They fundamentally make or break the practicality of this and other ideas that will look similar and ultimately decide whether the rendition of this idea gets applied or not by the people who have to rely on it. Too much syntax and you've just recreated a slightly simpler problem of having one instead of many languages, where nobody is very good at the language in terms of what you can do with it across the realms it can operate. The people using Python stay using it, and those using C stick with that – why switch if you're using it to do only the same things you're already doing now, and adding on learning tax? Conversely, too little syntax and you get something that is too weak to merit migration from the languages it's trying to combine and simplify. You can't be in some weird middle point between schema and logic – schema is the easy representation, logic needs the power and configurability where accessibility is a closer but still obvious afterthought in the immediate sense. Likewise, if runtime behaviors from a configurable system are not critically assessed, you get crashes and errors and bugs and pull requests and angry emails and more time spent later. You could have designed it correctly to begin with and dealt with far less as adoption grows and problems you didn't see suddenly become apparent. What settings best fit across all use cases has to also be taken into account. Some things are clear winners in the modern era – GC, async and non-blocking IO in general, multithreading and exception handling – while others aren't so clear cut when you have to do it all in terms of the web, desktop apps, low-level raw memory addressing, and others – stack size, safety, integer width, etc.

As mentioned before, it's a lot of iteration. Time is spent going over countless, random third-party libraries to ensure that you're getting code that works *in both directions*. That means converting to USAMS and immediately back to native and then compiling and testing it both builds and produces the same behavior and outputs for given inputs. Across all the target languages. That's not a weekend project. Thought and group input is really the best way to go about designing the syntax for the schema component. What looks good to one may look confusing or foreign to another. It'll be multiple people rather than any single individual or demographic that makes this work.

Contact / Feedback

Please feel free to reach out to me via email at albazzaztariq@gmail.com. Any feedback, concerns, helping hands, or otherwise are appreciated. This is a big project that entails a lot. I would love to spread this work and add inputs and brain power by involving more people, even better a whole small community. The good part is that conceptually it's actually not very deep. It's a time problem rather than a mind one. More people solving this means it gets done more quickly and the accessibility angle gets covered better by simply involving an array of people, particularly if they're the newer generation enticed by things like Claude Code.